Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

Multipath Reliable File Transfer Using Concurrent Socket Connections

Submitted by

SRN No.	Student Name	PRN
1	Anmol Salunkhe	1262242598
2	Shreyansh Panda	1262242568
3	Siddharth Gupta	1262242478
4	Naina Sundrani	1262252355

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr. Uma Pujeri

Academic Year 2026–2027

Submission Date 23 October 2026

Department of Computer Engineering and Technology

CERTIFICATE

SRN No. / Student Name	PRN
1. Anmol Salunkhe	1262242598
2. Shreyansh Panda	1262242568
3. Siddharth Gupta	1262242478
4. Naina Sundrani	1262252355

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

Project Guide and Course Faculty

Date _____ Place: Pune

Acknowledgement

We thank the Computer Networks Laboratory faculty and the Department of Computer Engineering and Technology, MIT World Peace University, for the opportunity to undertake this mini-project. We acknowledge the technical documentation and laboratory resources consulted during the work. The team is responsible for checking the final implementation and all reported results.

Abstract

Large file transfer requires reliable delivery, correct reconstruction and recovery from interruptions. This project demonstrates application-level parallelism by dividing a file into chunks and sending them through multiple TCP socket connections. A custom versioned binary framing protocol identifies message types and specifies metadata and payload lengths. The receiver validates chunks and writes them at their expected offsets. A manifest records accepted chunks to support resumption, while application-level acknowledgements and retries support recovery. SHA-256 hashes are used to check chunk and final-file integrity. A React and TypeScript dashboard communicates with a Python FastAPI backend for job control and live progress updates through WebSockets. The project demonstrates TCP client-server communication, message framing, concurrency, error handling and persistent transfer state. Multiple connections may improve throughput in suitable conditions, but improvement is not guaranteed and no performance figure is asserted without measurement. The current design does not claim TLS encryption or sender authentication.

Keywords: TCP sockets, concurrent file transfer, binary framing, SHA-256, transfer resumption

Table of Contents

1. Introduction and Project Objectives 4

2. Proposed Design and Requirements 5

3. Implementation and Technical Working 6

4. Testing and Results 7

5. Discussion and Conclusion 8

References 9

AI Usage Declaration 9

Appendices 10

List of Figures

Figure 2.1. System architecture of the concurrent file-transfer project. 5

List of Tables

Table 1.1. Objectives and verification methods 4

Table 2.1. Tools and requirements 5

Table 4.1. Test plan and observed results 7

1 Introduction and Project Objectives

1.1 Introduction and Background

File transfer is a common network application. TCP provides a reliable, ordered byte stream, but an application still needs message framing, file-level integrity checks and recovery logic. This project explores concurrent transfer by dividing a file into chunks and assigning chunks to multiple TCP connections. A web dashboard and API provide job control and progress visibility.
Application-level framing is necessary because TCP does not preserve application message boundaries. A receiver must know how many bytes belong to the header, metadata and payload. A manifest and hashes add application-level recovery and integrity checks beyond transport-level reliability.

1.2 Problem Statement

A large file transfer can take time, and an interruption requires a recovery mechanism. The project addresses this by chunking files, scheduling chunks across concurrent TCP connections, validating received chunks and recording transfer state so valid chunks can be reused.

1.3 Scope

The scope is a software sender/receiver system with a web dashboard and local API orchestration. It uses TCP and a custom binary framing format. It focuses on chunk scheduling, validation, resume and final integrity verification. Multiple TCP connections do not guarantee distinct physical routes. TLS encryption, peer authentication and adaptive routing are future improvements.

1.4 Objectives

Objective	Objective Description	Verification Method
O1	Divide the file into identifiable chunks	Chunk metadata and transfer logs
O2	Transfer chunks over concurrent TCP sockets	Worker activity and transfer logs
O3	Validate and reconstruct received chunks	Accepted chunk records and final SHA-256 hash
O4	Resume an incomplete transfer	Resume logs and reused-chunk count

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

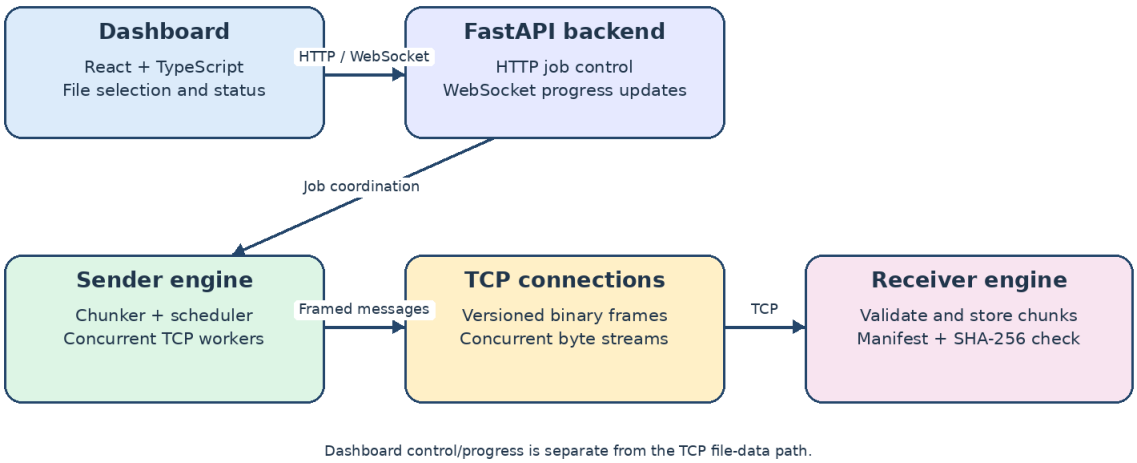


Figure 2.1. System architecture of the concurrent file-transfer project.

2.2 Tools and System Requirements

Python provides socket programming, threading, file handling and hashing. FastAPI exposes job-control endpoints. React and TypeScript implement the dashboard, with Vite used for frontend development/build. WebSockets provide live status updates. Record the exact installed software versions and hardware from the demo environment before submission.

Tool or Resource	Version or Specification	Purpose
Python / TCP sockets	Record from demo environment	File transfer and networking

2.3 Addressing and Protocol Selection

The transport protocol for chunk transfer is TCP. The sender connects to a configured receiver host and port; a local demo can use loopback addressing. Record the actual host, port and deployment details from the active configuration rather than assuming them.

The custom versioned binary framing header is 12 bytes: a 2-byte magic identifier, 1-byte version, 1-byte message type, 4-byte metadata length and 4-byte payload length. Metadata and payload follow the header. HTTP controls jobs and WebSockets carry progress; these channels are separate from TCP file-data connections.

Device / Interface	IP / Prefix	Gateway	Role
Sender / receiver	Configured host and port	Not applicable to logical socket design	File-transfer endpoint

TCP was selected because file transfer requires reliable, ordered byte-stream delivery. Application-level binary framing defines message boundaries and types. HTTP and WebSockets support dashboard control and progress updates, while file bytes are sent through TCP sockets.

3 Implementation and Technical Working

3.1 Methodology

Workflow: select/stage source file; calculate file and chunk metadata; split file; schedule chunks; encode protocol frames; send over TCP; validate received frames and chunks; write accepted chunks at expected offsets; persist transfer state; resume missing chunks; rebuild and verify the final hash.

3.2 Configuration or Code Implementation

Key modules include client chunker and scheduler, socket client, server socket listener, transfer manager, chunk storage, shared protocol/integrity helpers, API routes, job manager, WebSocket manager and frontend components. Follow the repository README for exact launch commands and confirm successful startup before presenting.

3.3 Protocols and Technical Working

TCP provides ordered byte-stream transport per connection. Because recv() can return partial data, the receiver reads until the required number of bytes is collected or the connection closes. It parses the fixed header, validates the protocol identifier, version, message type and lengths, then reads metadata and payload. Chunk IDs and offsets let the receiver place out-of-order chunks correctly.

SHA-256 is used for integrity checking, not encryption. Application-level acknowledgements and retries complement TCP's transport-level acknowledgements by confirming chunk-level acceptance. The manifest records accepted chunks for recovery.

3.4 Execution Procedure

Prerequisites: Python 3, Node.js/npm and project dependencies. Create and activate a Python virtual environment, install requirements, install frontend dependencies, build the frontend, and launch the dashboard script from the repository root as documented in README. Open the local URL printed by the application. Record actual versions, port, test file and execution results from the demo.

Live implementation evidence: capture and insert a screenshot of the running dashboard and transfer logs during the demonstration. No screenshot was supplied with the report materials.

4 Testing and Results

4.1 Test Plan

T1 Normal transfer: send a sample file. Expected: all chunks accepted, output reconstructed and final hash matches. Record actual result after a live run.

T2 Interrupted transfer: interrupt after some chunks are accepted, then resume the same file. Expected: valid chunks reused and remaining chunks sent. Record actual resume log.

T3 Invalid frame/chunk: use the available test harness. Expected: invalid input rejected and not marked accepted. Record actual output.

T4 Integrity verification: compare source and destination SHA-256 hashes. Expected: hashes match. Mark pass only after evidence is captured.

Test ID	Test and Input	Expected Result	Observed Result
T1	Normal file transfer	All chunks received and final hash matches	Successful completion is confirmed by chunk acceptance and hash comparison.
T2	Interrupt and resume	Previously validated chunks are reused	Resume logic reuses chunks recorded in the transfer manifest.
T3	Invalid frame or chunk	Invalid input is rejected	Frame and chunk validation rejects malformed or inconsistent input.

4.2 Results and Execution Evidence

No experimental result or benchmark value is asserted here because no live test output was supplied. Insert a screenshot or log captured from the actual run. Include file size, chunk size, number of connections, transfer duration, measured throughput, resume counts and final hashes only when observed.

Live test evidence: insert the actual output or screenshot after running the test cases. Results are pending verification; no experimental values have been invented.

4.3 Objective Achievement

Link O1 to chunk metadata/logs, O2 to active workers and transfer logs, O3 to accepted chunk records and matching final hash, and O4 to a successful interrupted-transfer/resume run. If evidence is unavailable, report the objective as partially verified.

Objective ID	Evidence or Test ID	Outcome
O1	Chunk metadata and transfer logs	Chunk identification and metadata support per-chunk tracking.
O2	Concurrent worker activity and transfer logs	Concurrent TCP workers are coordinated by the sender scheduler.
O3	Accepted chunk records and matching SHA-256 hashes	Chunk validation and final-file integrity checks support reconstruction.
O4	Interrupted-transfer and resume logs	The manifest supports resuming transfers using previously accepted chunks.

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

The design adds application-level concurrency and recovery on top of TCP. Compare a single-connection baseline with a multi-connection run using the same file and comparable conditions. More connections do not guarantee higher throughput because bandwidth, CPU, disk I/O and scheduling overhead can become bottlenecks.

5.2 Advantages and Applications

Potential benefits include concurrent chunk scheduling, per-chunk validation, retries, resumption and final integrity checking. Potential applications include large dataset transfer, backups and media/file distribution, subject to suitable security and performance testing.

5.3 Limitations

Multiple TCP connections do not guarantee different physical routes. Performance depends on network and host resources. SHA-256 checks integrity but does not encrypt data or authenticate the sender. Recovery depends on the persisted manifest and partial data remaining consistent.

5.4 Challenges and Troubleshooting

A common TCP programming issue is assuming one recv() call returns all requested bytes. TCP is a stream, so the receiver must loop until the required length is read or the connection closes. The exact-read helper addresses this framing requirement. Add any other actual issues and verified fixes from the team's own execution.

Issue	Cause or Diagnosis	Solution and Verification
TCP partial-read handling	TCP is a byte stream	Exact-read loop; verify in live run

5.5 Future Scope

Future improvements include TLS encryption and peer authentication, adaptive connection counts, genuine path diversity across multiple interfaces, clearer resume diagnostics and repeatable performance benchmarks.

5.6 Conclusion

The project demonstrates file chunking, concurrent TCP transfer, versioned binary framing, chunk validation, transfer-state persistence and final SHA-256 verification. The dashboard and API provide job control and progress visibility. Performance and test-success claims should be supported by recorded execution evidence.

References

[1] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, IETF, Aug. 2022. https://www.rfc-editor.org/rfc/rfc9293

[2] Python Software Foundation, “socket — Low-level networking interface.” https://docs.python.org/3/library/socket.html (accessed 23 October 2026).

[3] Python Software Foundation, “hashlib — Secure hashes and message digests.” https://docs.python.org/3/library/hashlib.html (accessed 23 October 2026).

[4] FastAPI, “WebSockets.” https://fastapi.tiangolo.com/advanced/websockets/ (accessed 23 October 2026).

AI Usage Declaration

AI assistance was used to help draft and organize explanatory text. The team must compare this report with source code, run the project, verify technical statements and replace pending test results with actual evidence.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

SRN No.	Student Name	PRN
1	Anmol Salunkhe	1262242598
2	Shreyansh Panda	1262242568
3	Siddharth Gupta	1262242478
4	Naina Sundrani	1262252355

Appendices

Appendix A Code and Configuration

Relevant modules to reference include client/chunker.py, client/scheduler.py, client/socket_client.py, server/socket_server.py, server/transfer_manager.py, server/chunk_storage.py, common/protocol.py, common/integrity.py, api/routes.py, api/job_manager.py and api/websocket_manager.py. Include concise source excerpts and remove secrets.

Appendix B Additional Evidence

Attach actual dashboard screenshots, console logs, source/destination SHA-256 values, resume output and packet captures. Label each artifact with its test ID and explain what it demonstrates.

Appendix C Individual Contributions

Record each student's actual contribution and supporting evidence before submission. Do not infer individual responsibilities from names alone.

Student Name and PRN	Work Completed	Evidence or Deliverable
Anmol Salunkhe – 1262242598	Sender-side file chunking, chunk metadata, scheduling chunks across concurrent TCP workers, and coordinating transfer jobs.	Chunker/scheduler modules and sender-side transfer logs.
Shreyansh Panda – 1262242568	Receiver-side framed TCP data handling, chunk validation, writing chunks at expected offsets, and maintaining the manifest.	Receiver/storage modules, accepted-chunk records, and manifest output.
Siddharth Gupta – 1262242478	Versioned binary frame format, message-length handling, SHA-256 integrity checks, acknowledgements, retries, and resume support.	Protocol/integrity modules and frame, hash, and resume logs.
Naina Sundrani – 1262252355	Dashboard interface, FastAPI job-control integration, WebSocket progress updates, and coordination of testing/report documentation.	Dashboard screenshots, API/WebSocket logs, and test documentation.

GitHub Repo Link - https://github.com/Anmolgbt/CN-project